

# Description Logics and the Semantic Web

Umberto Straccia

ISTI - CNR,

<http://gaia.isti.cnr.it/~straccia>

[straccia@isti.cnr.it](mailto:straccia@isti.cnr.it)

2006

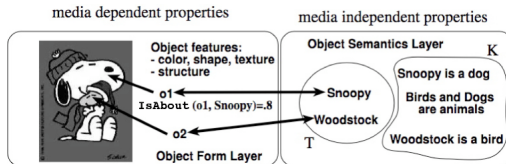
# Exercise

**Problem:** Using Protege, try to encode the web site `www.autos.com` as ontology, starting from the lecture slides “Part III”

**Solution:** A partial solution is give in the file “autos.owl”

# Exercise

**Problem:** Encode the following scenario in Description Logics and write the query “find images having an image region, which is about animals” as concept



**Solution:**

$$\begin{aligned} \text{Image} &\sqsubseteq (\exists \text{hasColor}.\text{Color}) \\ &\sqcap (\exists \text{hasShape}.\text{Shape}) \\ &\sqcap (\exists \text{hasTextures}.\text{Texture}) \\ &\sqcap (\exists \text{hasImageRegion}.\text{ImageRegion}) \end{aligned}$$

$$\text{Bird} \sqsubseteq \text{Animal} \quad \text{Dog} \sqsubseteq \text{Animal} \quad \text{Animal} \sqsubseteq \text{LivingThing}$$

snoopy:Dog  
woodstock:Bird

$$\begin{aligned} (o_1, \text{snoopy}) &:\text{IsAbout} & (o_2, \text{woodstock}) &:\text{IsAbout} & o_1 &:\text{ImageRegion} \\ o_2 &:\text{ImageRegion} & i &:\text{Image} & (i, o_1) &:\text{hasImageRegion} \\ (i, o_2) &:\text{hasImageRegion} \end{aligned}$$

$$\text{QueryConcept} = \text{Image} \sqcap (\exists \text{hasImageRegion} . (\exists \text{IsAbout} . \text{Animal}))$$

# Exercise

**Problem:** Try to encode in description logics the following definitions

- 1 Pizza, Topping and PizzaBase are disjunct concepts/classes
- 2 A cheese pizza is defined as a pizza, having a topping and having a pizza base. The topping is a cheese topping, while the base is a PizzaBase. A cheese topping is a topping.
- 3 The domain of the relation topping is pizza, while the range is topping

**Solution:**

- 1 We use three inclusion axioms:

$$Pizza \sqsubseteq \neg Topping$$

$$Pizza \sqsubseteq \neg PizzaBase$$

$$Topping \sqsubseteq \neg PizzaBase$$

- 2

$$CheesePizza = Pizza$$

$$\sqcap (\exists hasTopping. CheeseTopping)$$

$$\sqcap (\exists hasPizzaBase. PizzaBase)$$

$$CheeseTopping \sqsubseteq Topping$$

- 3 We use the following axioms. The first defines the domain, the second defines the range.

$$\exists hasTopping. \top \sqsubseteq Pizza$$

$$\top \sqsubseteq \forall hasTopping. PizzaTopping$$



# Exercise

**Problem:** Translate the following Description Logic knowledge base into First-Order-Logic

|                                     |               |                                                                |
|-------------------------------------|---------------|----------------------------------------------------------------|
| <i>Pizza</i>                        | $\sqsubseteq$ | $\neg \textit{Topping}$                                        |
| <i>Pizza</i>                        | $\sqsubseteq$ | $\neg \textit{PizzaBase}$                                      |
| <i>Topping</i>                      | $\sqsubseteq$ | $\neg \textit{PizzaBase}$                                      |
| <i>CheesePizza</i>                  | $=$           | <i>Pizza</i>                                                   |
|                                     |               | $\sqcap (\exists \textit{hasTopping}. \textit{CheeseTopping})$ |
|                                     |               | $\sqcap (\exists \textit{hasPizzaBase}. \textit{PizzaBase})$   |
| <i>CheeseTopping</i>                | $\sqsubseteq$ | <i>Topping</i>                                                 |
| $\exists \textit{hasTopping}. \top$ | $\sqsubseteq$ | <i>Pizza</i>                                                   |
| $\top$                              | $\sqsubseteq$ | $\forall \textit{hasTopping}. \textit{PizzaTopping}$           |

**Solution:**

|                                                                         |                   |                                                                                   |
|-------------------------------------------------------------------------|-------------------|-----------------------------------------------------------------------------------|
| $\forall x \textit{Pizza}(x)$                                           | $\Rightarrow$     | $\neg \textit{Topping}(x)$                                                        |
| $\forall x \textit{Pizza}(x)$                                           | $\Rightarrow$     | $\neg \textit{PizzaBase}(x)$                                                      |
| $\forall x \textit{Topping}(x)$                                         | $\Rightarrow$     | $\neg \textit{PizzaBase}(x)$                                                      |
| $\forall x \textit{CheesePizza}(x)$                                     | $\Leftrightarrow$ | $(\textit{Pizza}(x)$                                                              |
|                                                                         |                   | $\wedge (\exists y \textit{hasTopping}(x, y) \wedge \textit{CheeseTopping}(y))$   |
|                                                                         |                   | $\wedge (\exists y \textit{hasPizzaBase}(x, y) \wedge \textit{PizzaBase}(x, y)))$ |
| $\forall x \textit{CheeseTopping}(x)$                                   | $\Rightarrow$     | <i>Topping</i> (x)                                                                |
| $\forall x \exists y \textit{hasTopping}(x, y) \wedge \textit{True}(y)$ | $\Rightarrow$     | <i>Pizza</i> (x)                                                                  |
| $\forall x \textit{True}(x)$                                            | $\Rightarrow$     | $(\forall y \textit{hasTopping}(x, y) \Rightarrow \textit{PizzaTopping}(y))$      |

## Exercise

**Problem:** Translate the following Description Logic knowledge base into LISP-like syntax of the reasoner fuzzyDL  
<http://gaia.isti.cnr.it/~straccia/software/fuzzyDL/fuzzyDL.html>

|                                      |               |                                                                                                                                                 |
|--------------------------------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Pizza</i>                         | $\sqsubseteq$ | $\neg \textit{Topping}$                                                                                                                         |
| <i>Pizza</i>                         | $\sqsubseteq$ | $\neg \textit{PizzaBase}$                                                                                                                       |
| <i>Topping</i>                       | $\sqsubseteq$ | $\neg \textit{PizzaBase}$                                                                                                                       |
| <i>CheesePizza</i>                   | $=$           | $\textit{Pizza} \sqcap (\exists \textit{hasTopping} . \textit{CheeseTopping})$<br>$\sqcap (\exists \textit{hasPizzaBase} . \textit{PizzaBase})$ |
| <i>CheeseTopping</i>                 | $\sqsubseteq$ | <i>Topping</i>                                                                                                                                  |
| $\exists \textit{hasTopping} . \top$ | $\sqsubseteq$ | <i>Pizza</i>                                                                                                                                    |
| $\top$                               | $\sqsubseteq$ | $\forall \textit{hasTopping} . \textit{PizzaTopping}$                                                                                           |

*p:Pizza t:CheeseTopping pb:PizzaBase (p, t):hasTopping (p, pb):hasPizzaBase*

**Solution:**

(disjoint Pizza Topping PizzaBase)

```
(define-concept CheesePizza (and Pizza
  (some hasTopping CheeseTopping)
  (some hasPizzaBase PizzaBase)))
```

(implies CheeseTopping Topping)

```
(domain hasTopping Pizza)
(range hasTopping PizzaTopping)
```

```
(instance p Pizza) (instance t CheeseTopping) (instance pb PizzaBase)
(related p t hasTopping) (related p pb hasPizzaBase)
```

# Exercise

**Problem:** Verify that the individual “p” of the previous example is a cheese pizza

**Solution:** Submit the following file to the fuzzyDL reasoner, by selecting the reasoning mode “classical”  
(disjoint Pizza Topping PizzaBase)

```
(define-concept CheesePizza (and Pizza
                                   (some hasTopping CheeseTopping)
                                   (some hasPizzaBase PizzaBase)))

(implies CheeseTopping Topping)

(domain hasTopping Pizza)
(range hasTopping PizzaTopping)

(instance p Pizza) (instance t CheeseTopping) (instance pb PizzaBase)
(related p t hasTopping) (related p pb hasPizzaBase)

(individual-instance? p CheesePizza)
```

